

AI SDLC Course 101 Assignment - 1

Exercise-Based Learning - Participant Guide

Problem: Personal Bookmark Manager with Tagging

1. Objective

The objective of this exercise is to practice using GitHub Copilot or another AI coding assistant throughout the Software Development Life Cycle (SDLC). The focus is not only on generating code. You should use AI to help understand requirements, plan the solution, make design decisions, implement features, test the application, review the work, and improve documentation.

You are expected to use AI extensively, but sensibly. Good AI-assisted development means providing useful context, evaluating suggestions, making your own decisions, verifying outputs, and iterating when the first answer is not good enough.

2. What you need to do

- Build the Personal Bookmark Manager described in this document.
- Run the application locally on your machine and verify the required functionality.
- Use GitHub Copilot or another approved coding agent during all major SDLC phases.
- Maintain the required Markdown documentation as you work.
- Record a short final demo video showing the working application and selected examples of how AI assisted you.
- Submit the required Markdown files and demo video according to the submission guidelines.

3. Problem Statement

Personal Bookmark Manager with Tagging

Develop a personal bookmark manager that enables users to save URLs, assign tags, and search or filter their stored bookmarks. The application should help users efficiently organize web resources they encounter during research or everyday browsing.

Functional Requirements

Requirement	Acceptance Criteria
Add Bookmark	User can submit a URL with an optional title. If no title is provided, the application should attempt to retrieve a useful page title. If retrieval fails, handle it gracefully.
Tag Bookmarks	User can assign one or more tags to a bookmark.
List Bookmarks	User can view all saved bookmarks, with the most recent entries first.
Filter by Tag	User can select a tag and see only bookmarks associated with that tag.

Search	User can search bookmarks by title or URL.
Edit Bookmark	User can change the URL, title, or tags of an existing bookmark.
Delete Bookmark	User can remove a bookmark. Provide a reasonable confirmation or safeguard against accidental deletion.
Persistence	Bookmarks remain available after the application is stopped and restarted.
Validation	Clearly invalid or empty URLs should not be saved; show understandable feedback.
Duplicate Handling	The application should detect or sensibly handle attempts to save the same URL more than once.
Empty/Error States	Handle cases such as no bookmarks, no search results, title-fetch failure, and invalid input in a user-friendly way.

Technical Constraints

- The application must run locally and provide a web-based user interface accessible through a browser.
- Technology stack is flexible. Choose a stack that you can work with and that is appropriate for the problem.
- No paid cloud service or external database is required.
- The final application must remain runnable on your machine during the evaluation period.
- Docker/deployment is not required for this version of the exercise unless separately instructed.

Keep Files in Sync

You write these files as you work, so later phases will change earlier decisions. When that happens:

- **Update the earlier file** so it reflects your final position, for example a changed data model goes back into 02-design.md.
- **Record what changed and why** in the phase where the change came up, such as Implementation Plan vs. Actual in 03-build.md or Findings in 05-review.md.
- **Never leave two files disagreeing** about the same decision, data model, or requirement status.

Confidentiality, Data & Licensing

Treat everything you submit as material that will be read outside your team.

- **No personal data.** Use synthetic test data only. No real names, emails, phone numbers, addresses, or account details in examples, test data, screenshots, or logs. Redact anything personal that appears in a screenshot.
- **No client or employer material.** No client code, client names, internal URLs, internal system names, proprietary data, or copied internal documentation.
- **No secrets.** No API keys, tokens, passwords, or connection strings in text, screenshots, or logs.
- **Properly licensed tools only.** Prefer open-source or free-to-use tools, libraries, and plugins. Never use a paid tool without a valid license for this use — no unlicensed, cracked, or expired-trial software.
- **No copied proprietary code.** If an AI suggestion appears to reproduce licensed or proprietary code, do not use it — ask for an original implementation instead.

Optional Enhancements

After completing the mandatory requirements, you may add useful enhancements such as import/export, favorites, automatic tag suggestions, broken-link checking, responsive UI, accessibility improvements, dark mode, or related-link search. Complete the core requirements first.

4. How to use AI during the SDLC

Planning: Use AI to understand and break down requirements, create user stories and acceptance criteria, identify edge cases, estimate effort, identify risks, and raise questions or assumptions.

Design: Use AI to explore architecture/project structure, data models, UI components/user flow, technology choices, validation, and error-handling strategies. Ask for alternatives and trade-offs rather than accepting the first design.

Build: Use AI for implementation guidance, code generation/completion, boilerplate, CRUD logic, validation, debugging, and explaining unfamiliar code. Give AI the relevant requirement, current context, errors, and constraints.

Testing: Use AI to generate test scenarios and edge cases, improve test coverage, investigate failures, and suggest additional checks. Run the tests/checks yourself and record actual outcomes.

Review: Use AI to review the solution for correctness, security, maintainability, validation, accessibility, UX, and possible defects. Decide which suggestions are relevant and verify changes.

Reflection & Documentation: Use AI to help organize your notes if useful, but ensure all claims, decisions, outcomes, and evidence represent your actual work.

5. Important guidelines for working with AI

- Do not use AI only as a code generator. Use it across planning, design, build, testing, and review.
- Provide context. A good request normally includes what you are trying to do, relevant requirements, constraints, current behavior/code context, errors, and what you have already tried.
- Do not blindly accept suggestions. Read them, question them, modify or reject them when appropriate, and verify the result.
- Iteration is encouraged. If the first response is incomplete or incorrect, refine the context or prompt and continue.
- Record important interactions while you work rather than trying to reconstruct everything at the end.
- Failures are useful evidence. If an AI suggestion did not work, document what happened, how you diagnosed it, and what you did next.
- Do not manufacture rejected suggestions. If a recommendation was good and you accepted it, say so.
- Do not paste very large code blocks or entire AI conversations into the documentation. Capture the important interaction and its impact.
- Clearly distinguish between what AI suggested and what you decided or changed.
- Do not claim that something works unless you actually verified it.

6. Required submission structure

Create a docs folder containing the following six Markdown files:

File	What it should contain
docs/01-planning.md	Problem understanding, requirement breakdown, user stories, edge cases, risks, planning decisions, and important AI interactions.
docs/02-design.md	Solution/design, data model, UI flow, error handling, alternatives/trade-offs, and important AI interactions.
docs/03-build.md	Feature implementation journey, AI-assisted development, troubleshooting, human changes/decisions, and local verification.
docs/04-testing.md	Test strategy, test matrix, AI-assisted test generation, failures/fixes/retesting, and known limitations.
docs/05-review.md	AI-assisted review findings, accepted/modified/rejected recommendations, fixes, false positives, and final readiness review.
docs/06-reflection.md	Overall AI usage, approximate time/effort, rework, important human decisions, learning, and demo-video details.

7. Standard format for important AI interactions

Use the following format whenever you document an important AI interaction. You do not need to document every prompt. Select interactions that materially influenced a decision, feature, test, fix, or review outcome.

A record counts only if it shows:

1. **Context:** the requirement, code, error, or constraint you gave the AI.
2. **Decision:** Accepted, Modified, or Rejected, and why.
3. **Verification:** what you ran, tested, or inspected.

Routine completions and one-line answers don't count.

Example. Put a blank line between fields so they render on separate lines.

```
### Evidence E-<phase>-<number>

**SDLC activity:** <planning/design/build/test/review/documentation>

**Task/feature:** <what you were trying to accomplish>

**Context given to AI:** <requirements, code context, error, constraints, etc.>

**Prompt/request:** <what you asked>

**AI response summary:** <what AI suggested>

**Your decision:** <Accepted / Modified / Rejected>

**What you changed and why:** <your reasoning>

**How you verified it:** <test/run/inspection/comparison/etc.>
```

****Outcome:**** <worked / partially worked / failed / caused rework>

****Iteration:**** <what you changed or asked next, if applicable>

****Approx. time:**** <minutes spent on this activity>

****Learning:**** <one concise takeaway>

Avoid: vague context, decisions without reasons, unverified claims, invented rejections, pasted conversations or large code blocks, and hiding suggestions that failed. Record failures honestly; they are useful evidence.

8. What to include in each Markdown File (Mandatory sections by File)

Use each section name below as a **##** heading, spelled exactly as listed and in the same order. If a section does not apply, keep the heading and say why.

01-planning.md

Section	Mandatory Requirements
Problem Understanding	The problem, target user, and value delivered, in your own words
Requirement Breakdown	The work split into logical features and tasks, with dependencies between them
User Stories & Acceptance Criteria	At least 4 meaningful user stories in "As a... I want... so that..." form, each with testable acceptance criteria
Edge Cases	At least 6 relevant edge cases (e.g. invalid URL, duplicate URL, title-fetch failure, no bookmarks, no search results, empty input), noting where AI helped find them
Non-Functional Requirements	Measurable targets covering at least: performance at a realistic data volume (e.g. "search returns in under 500 ms with 1,000 bookmarks"), reliability and persistence, usability and accessibility, and security
Risks, Assumptions & Questions	Important risks and uncertainties, assumptions made, and questions raised
AI Interactions	At least 3 evidence records in the format from section 7 of the Participant Guide (see Section 2.2)
Planning Outcome	What changed or improved in your plan after using AI

02-design.md

Section	Mandatory Requirements
Proposed Solution	Architecture and project structure; technology choices with the reason each suits this problem
Data Model	Entities, fields, and relationships, or equivalent persistence design; how bookmarks survive an application restart
UI / User Flow	Screens or components and the major user flow. Mermaid diagrams are encouraged
Error Handling	How validation and error cases are handled: invalid or empty URL, duplicate URL, title-fetch failure, empty states, and the feedback the user sees

Security Design	How untrusted input is handled: URLs submitted by the user, page titles fetched from external sites, and text used in search and tag queries. Where each is validated, escaped, or restricted; how dependencies are kept safe
Meeting the Non-Functional Requirements	How the design meets each target in 01-planning.md: data volume and query approach (indexes, pagination), persistence and recovery, and accessibility
Alternatives & Trade-offs	At least 2 meaningful design choices, each with the options considered, the decision, and its trade-offs
AI Interactions	At least 3 evidence records in the format from section 7 of the Participant Guide (see Section 2.2)
Design Verification	How you checked the design covers every mandatory functional requirement

03-build.md

Section	Mandatory Requirements
Implementation Plan vs. Actual	What you intended to build, what you actually built, and the important deviations with reasons
Feature Evidence Matrix	A table with one row for each mandatory functional requirement (Add Bookmark, Tags, List/Newest First, Filter by Tag, Search, Edit, Delete, Persistence, Validation, Duplicate Handling, Empty/Error States) and the columns: Requirement, Status, How AI helped, What I changed/decided, How I verified
Troubleshooting	At least 2 meaningful problems or errors: what happened, how you diagnosed it, and how you resolved it
Significant Human Changes	Examples where you corrected, simplified, modified, or rejected AI output, and why your version was better
AI Interactions	At least 5 evidence records in the format from section 7 of the Participant Guide (see Section 2.2), covering different features and problems
Local Run Evidence	How you ran the application and verified the major functionality, with screenshots. No large code blocks

04-testing.md

Section	Mandatory Requirements
Test Strategy	What you decided to test and why; what is automated vs manual; the tools used; at least one check against each non-functional target, including a security check on untrusted input
Test Matrix	Table with: requirement, scenario, expected result, actual result , pass/fail, and whether AI helped create the test. Cover the happy path, validation, duplicates, and empty states
AI-Discovered Edge Cases	Useful scenarios the AI raised that you had not considered, and their results
Fail -> Fix -> Retest	At least 2 examples if failures occurred: the failure, the fix, and the retest result. If fewer than 2 failures occurred, document the ones that did and say so. Never invent failures
AI Interactions	At least 3 evidence records in the format from section 7 of the Participant Guide (see Section 2.2)
Known Limitations	Incomplete tests, unresolved issues, and untested areas, stated honestly

05-review.md

Section	Mandatory Requirements
Review Scope	Which areas you asked the AI to review: correctness, security, maintainability, accessibility, validation, UX
Findings	Table with: finding, importance/severity, your assessment, action taken, and how you verified it
Accepted Feedback	Recommendations that improved the solution, and the improvement each made
Modified/Rejected Feedback	Recommendations you changed or did not follow, with reasoning. If there were none, say so honestly
False Positives / Misses	Anything the AI misunderstood, or real issues it failed to spot, if applicable
AI Interactions	At least 3 evidence records in the format from section 7 of the Participant Guide (see Section 2.2)
Final Readiness Check	Every mandatory functional requirement and non-functional target mapped to the final solution, with its status

06-reflection.md

Section	Mandatory Requirements
AI Usage Summary	Where AI was most useful and least useful, with examples
Approximate SDLC Time	Time spent on planning, design, build, testing, and review. Estimates are fine
Rework	Where AI saved time, and where it created extra work
Human Judgment	3 decisions where your own judgment materially mattered, and what would have happened otherwise
What You Would Do Differently	Improvements to your prompts, the context you gave, or your process
Self-Assessment	Your strongest phase, weakest phase, and confidence in the result
Demo Video	Filename or link, and duration (5–8 minutes). A link must be accessible to reviewers
Declaration	Confirmation that the documentation reflects your actual work and AI interactions

9. Feature Evidence Matrix

Include a table similar to this in docs/03-build.md:

Requirement	Status	How AI helped	What I changed/decided	How I verified
Add Bookmark	<fill in>	<fill in>	<fill in>	<fill in>
Tags	<fill in>	<fill in>	<fill in>	<fill in>
List/Newest First	<fill in>	<fill in>	<fill in>	<fill in>
Filter by Tag	<fill in>	<fill in>	<fill in>	<fill in>
Search	<fill in>	<fill in>	<fill in>	<fill in>
Edit	<fill in>	<fill in>	<fill in>	<fill in>

Delete	<fill in>	<fill in>	<fill in>	<fill in>
Persistence	<fill in>	<fill in>	<fill in>	<fill in>
Validation	<fill in>	<fill in>	<fill in>	<fill in>
Duplicate Handling	<fill in>	<fill in>	<fill in>	<fill in>
Empty/Error States	<fill in>	<fill in>	<fill in>	<fill in>

10. Demo Video Guidelines

Record a short 5-8 minute video on your machine. This is a technical demonstration, not a presentation contest. Simple screen recording with your voice is sufficient.

- Briefly introduce yourself and show the running application.
- Add a bookmark and assign tags.
- Demonstrate listing, search, and tag filtering.
- Edit and delete a bookmark.
- Show at least one validation or edge-case behavior.
- Demonstrate that saved bookmarks persist after restarting the application, where practical.
- Show one useful AI interaction and explain what you did with the suggestion.
- Explain one problem, bug, or difficult point and how AI helped you work through it.
- Briefly state where AI was most useful and where your own judgment was most important.

11. Suggested Working Approach

Step 1 - Understand before building: Read the complete problem. Use AI to clarify requirements, identify missing cases, and turn the requirements into a plan.

Step 2 - Design a simple solution: Choose an appropriate stack and data model. Avoid unnecessary complexity. Ask AI for alternatives and trade-offs.

Step 3 - Build incrementally: Implement one requirement at a time. Run the application frequently. Document important AI interactions as they occur.

Step 4 - Test beyond the happy path: Use AI to challenge your solution with invalid inputs, duplicates, empty states, errors, and other edge cases.

Step 5 - Ask AI to review your work: Request targeted reviews for correctness, security, maintainability, UX, accessibility, and validation. Evaluate the suggestions yourself.

Step 6 - Complete documentation and demo: Check the six Markdown files for completeness and consistency, then record the final demo.

12. Submission checklist

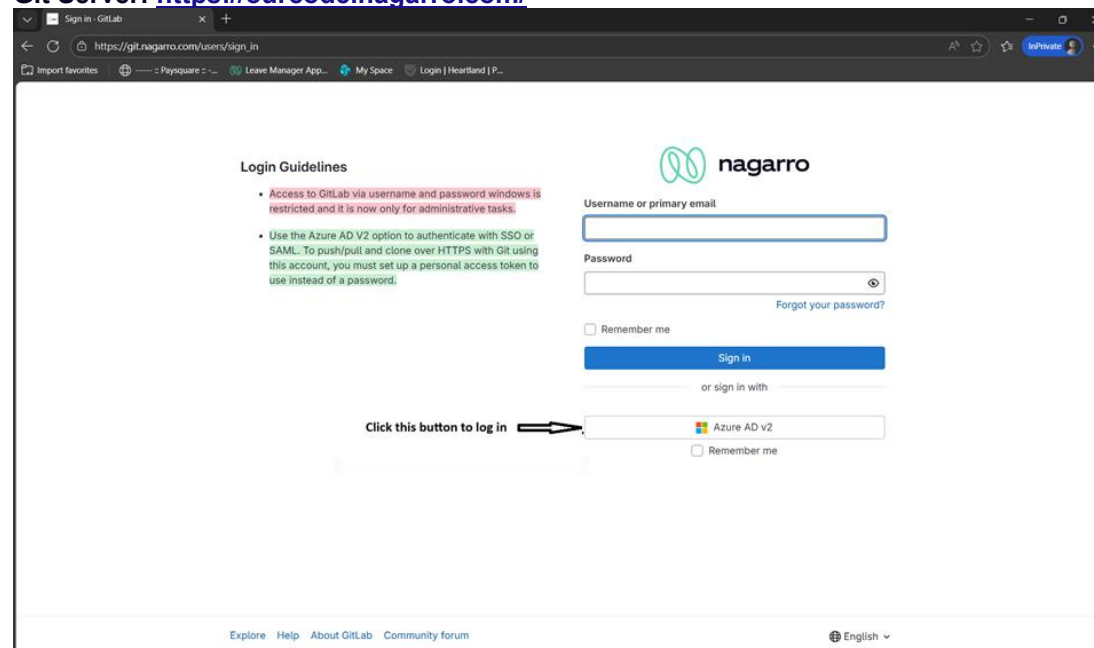
- ☐ All mandatory Bookmark Manager requirements are implemented or any incomplete requirement is clearly identified.
- ☐ Application runs locally on your machine.

- ☐ docs/01-planning.md completed.
- ☐ docs/02-design.md completed.
- ☐ docs/03-build.md completed, including Feature Evidence Matrix.
- ☐ docs/04-testing.md completed with actual test outcomes.
- ☐ docs/05-review.md completed.
- ☐ docs/06-reflection.md completed.
- ☐ Important AI interactions use the standard evidence format.
- ☐ Demo video recorded and accessible.
- ☐ Documentation reflects actual work and does not contain fabricated interactions, tests, or outcomes.
- ☐ Keep your local application/project available until the exercise review period is complete.

13. Submission Logistics

Repository: You will be granted access to one Git repository containing a predefined base folder structure separately.

Git Server: <https://ourcode.nagarro.com/>



Repository Structure: Use the shared starter template as your base. Do not restructure the top-level layout and keep your docs/ folder at the root as specified in Section 6.

Branch: Push your work directly to the main branch of your assigned repository, no feature branches or pull requests required for this exercise. Ensure your final commit is pushed before the submission deadline; commits after the deadline will not be considered unless an extension has been granted.

Documentation Format: Every AI interaction you document across all docs/*.md files must follow the standard Evidence format defined in Section 7 (Evidence E-<phase>-<number>).

Entries not following this format will not count toward the minimum evidence requirements per file.

What to submit: Only your docs/ folder (the six Markdown files) and your demo video link need to be submitted. You do **not** need to submit your source code separately from your local machine, but your project must remain runnable locally through the review period in case a spot-check or live walkthrough is requested (see Section 12).

14. Final Reminder

The purpose of this exercise is to learn how to work effectively with AI as part of software engineering. A successful submission is not the one with the largest application or the greatest number of AI prompts. Focus on using AI thoughtfully, making good engineering decisions, validating what it produces, and learning how AI can improve your SDLC workflow.